

Trees, Ensembles, Feature Engineering

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



موهبة
Mawhiba

KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 2, Day 12, Self-study

30 September 2026

1. Where This Fits
2. Learning Outcomes
3. Trees in Depth
4. Ensembles in Depth
5. Feature Engineering
6. Olympiad Practice
7. Check Yourself
8. Limitations and What Comes Next
9. Summary
10. References

- ▶ **In the lecture (Day 10):** a decision tree asks yes/no questions chosen by Gini impurity; `max_depth` is its overfitting dial; a random forest averages hundreds of random trees and gradient boosting chains trees that fix each other's errors; `GridSearchCV` tunes the dials; feature engineering builds the columns a model can use.
- ▶ **This deck adds** how a *regression* tree chooses a split, the dials beyond depth, the forest's free validation score, how many boosting trees to use and how to let scikit-learn stop by itself, boosting that eats missing values and categories without any encoding, and the feature-engineering moves that real Olympiad solutions used, each measured on a table you know.
- ▶ **How to use it:** read a frame, run its code on the used cars, the Steam games, the admissions or the Adult census in Colab, and compare with the numbers printed here. Then answer the three questions at the end.

Where the lecture's version breaks

- ▶ Day 10's grid search tried 6 forest settings and 12 boosting settings. It never asked the two questions that cost the most time on Friday: *how many trees are enough* and *can I skip the encoding?*
- ▶ The Adult census has 3,419 missing cells and eight text columns. One-hot plus a forest: ROC-AUC 0.895. The same table fed raw to HistGradientBoostingClassifier: **0.921**, in 2.5 seconds, no encoding, no filling.
- ▶ The used-car make has 57 values. As one-hot columns a Ridge model reaches MAE 20,219; encoded as a frequency it stays at 27,757; the forest barely cares. The right encoding depends on the model, and only a measurement tells you.

After this deck you can

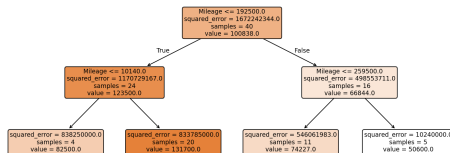
Read a regression tree, pick `min_samples_leaf` and `max_features`, get a validation score for free from the forest, choose the number of boosting trees from a curve or by early stopping, feed messy tables to boosting directly, and run the feature-engineering checklist that the NEOAI, HAO and ONIA reference solutions used.

By the end of this deck, you will be able to:

- ▶ **Explain** how a regression tree chooses a split (variance reduction) and read a fitted tree's thresholds and leaf values.
- ▶ **Tune** a tree with `min_samples_leaf` and a forest with `max_features`, and use the out-of-bag score as a free validation.
- ▶ **Choose** the number of boosting trees from a staged validation curve or with early stopping, and relate it to the learning rate.
- ▶ **Use** `HistGradientBoosting` on tables with missing values and categorical columns without encoding them.
- ▶ **Encode** a high-cardinality column three ways and measure which one each model prefers.
- ▶ **Build** leakage-free preprocessing with `ColumnTransformer` and `Pipeline`, and apply the feature checklist from real Olympiad solutions.

Trees & Ensembles: **Trees in Depth**

How a Regression Tree Chooses a Split

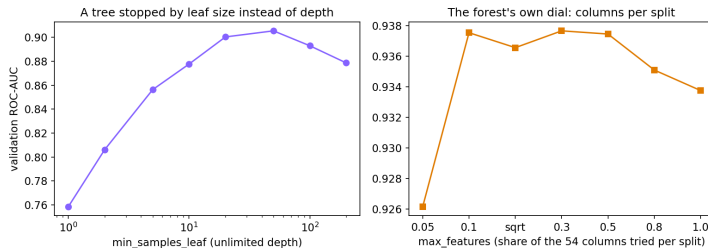


Source: the 40 training Tahoes of Day 09, `DecisionTreeRegressor(max_depth=2)` drawn with `plot_tree`; `scripts/day12.trees_figures.py`.

For a number target there is no Gini: each node predicts the **mean** of its cars, its impurity is the **MSE** around that mean, and the tree picks the cut that lowers the weighted MSE most.

The root split with numbers

40 cars, mean 100,838 SAR, $\text{MSE } 1.67 \times 10^9$. Cut at 192,500 km: left 24 cars, mean 123,500, $\text{MSE } 1.17 \times 10^9$; right 16 cars, mean 66,844, $\text{MSE } 4.99 \times 10^8$. Weighted: $(24 \cdot 1.17 + 16 \cdot 0.499) \times 10^9 / 40 = 9.02 \times 10^8$, a 46% drop. No other cut on this column drops it more.



Source: HAIO 2025 admissions data, 54 columns, 80/20 split; a tree with unlimited depth, a forest of 200 trees; scripts/day12_trees_figures.py.

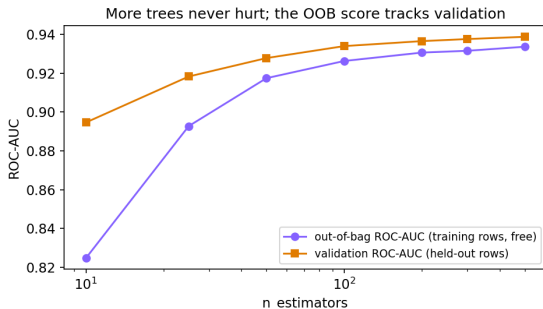
- ▶ **min_samples_leaf**: a leaf must hold at least this many rows. Unlimited depth with leaves of 50 scores ROC-AUC 0.905, better than Day 10's best depth (0.899): the tree is free to go deep where the data is dense and must stop where it is sparse.
- ▶ **max_features**: columns tried at each split of a forest. Too few (5% = 2 columns) starves the trees: 0.926. Anything from 10% to 50% gives 0.937; all columns (1.0) makes the trees alike and loses the averaging: 0.934. The default sqrt is a safe middle.

```
from sklearn.tree import DecisionTreeRegressor, export_text
tree = DecisionTreeRegressor(max_depth=2, random_state=0).fit(tr[["Mileage"]],
    tr["Price"])
print(export_text(tree, feature_names=["Mileage"], decimals=0))
# |--- Mileage <= 192500
# |   |--- Mileage <= 10140    -> value: [82500]
# |   |--- Mileage > 10140    -> value: [131700]
# |--- Mileage > 192500
# |   |--- Mileage <= 259500  -> value: [74227]
# |   |--- Mileage > 259500  -> value: [50600]
print(tree.tree_.threshold[0], tree.tree_.n_leaves)      # 192500.0  4
leaf_of_each_car = tree.apply(va[["Mileage"]])           # which leaf every
    validation car lands in
```

- ▶ `export_text` prints the rules; a depth-2 tree is four sentences a judge can read.
- ▶ The four leaf values are the only four prices this tree can ever predict: the staircase of Day 10. A car with 1,000,000 km still gets 50,600.
- ▶ Odd leaf: 4 cars under 10,140 km predicted at 82,500, below the 131,700 of the next leaf. Four rows decided it; this is the instability that forests exist to average out.

Trees & Ensembles: **Ensembles in Depth**

The Forest's Free Validation: Out-of-Bag



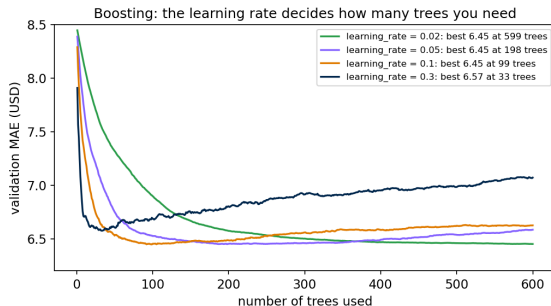
Source: admissions data, RandomForestClassifier(oob_score=True, warm_start=True)
grown from 10 to 500 trees; scripts/day12.trees.figures.py.

Each tree trains on a random sample *with replacement*, so about 37% of the rows are left out of it: its **out-of-bag** rows. Predicting every row only with the trees that never saw it gives a validation score without a split.

```
rf = RandomForestClassifier(300,  
                           oob_score=True,  
                           random_state  
                             =0)  
  
rf.fit(X_train, y_train)  
p_oob = rf.oob_decision_function[:, 1]  
roc_auc_score(y_train, p_oob) # 0.932
```

At 300 trees: OOB 0.932, held-out 0.938.
Past 200 trees the gain is in the third decimal: more trees never hurt, they only cost time.

Boosting: How Many Trees, and the Learning Rate



Source: ONIA Steam prices, 24 columns, GradientBoostingRegressor(max_depth=2), staged_predict on the 20% validation split; scripts/day12.trees_figures.py.

- ▶ staged_predict gives the prediction after 1, 2, ..., 600 trees in one pass, so one fit draws the whole curve.
- ▶ Rate 0.3: best after 33 trees, then the curve climbs: overfitting. Rate 0.1: best at 99 trees. Rate 0.05: 198. Rate 0.02: still falling at 600. All three slow rates reach 6.45; the fast one stops at 6.57.
- ▶ Rule: halve the rate, double the trees. Small rate plus many trees is the safe setting; the price is time.

```
curve = [mean_absolute_error(y_val, p)
         for p in gb.staged_predict(
             X_val)]
best_n = int(np.argmin(curve)) + 1
```

```
from sklearn.ensemble import HistGradientBoostingRegressor
hgb = HistGradientBoostingRegressor(learning_rate=0.05, max_depth=3, max_iter
    =1000,
    early_stopping=True, validation_fraction=0.15, n_iter_no_change=30,
    random_state=0)
hgb.fit(X_train, y_train)
print(hgb.n_iter_)                                # 111: it stopped by
    itself
print(mean_absolute_error(y_val, hgb.predict(X_val))) # 6.44
```

- ▶ HistGradientBoosting is scikit-learn's fast boosting (the same idea as LightGBM): it bins every column into 255 buckets first, so each tree is cheap and 1,000 trees take seconds.
- ▶ With `early_stopping=True` it keeps 15% of the training rows aside, scores them after every tree, and stops when 30 trees in a row bring no gain. Rate 0.05 stopped at 111 trees (MAE 6.44), rate 0.1 at 75 (6.42).
- ▶ The 15% comes out of the training rows, not out of your validation split, so your validation score stays honest.

The Adult census (Day 06): 32,537 rows, eight text columns, 3,419 cells that were "?". The usual route: replace, one-hot, fill, then fit. The short route:

```
adult = adult.replace("?", np.nan)
for c in adult.select_dtypes("object"):
    adult[c] = adult[c].astype("category")
hgb = HistGradientBoostingClassifier(
    categorical_features="from_dtype",
    random_state=0)
hgb.fit(X_train, y_train)
```

NaN goes down whichever branch helped on the training rows (a learned “missing” direction); a category column is split as a set (“workclass in {Private, Self-emp}”), no one-hot needed.

Adult, 20% validation	ROC-AUC	accuracy
one-hot (105 cols) + logistic	0.900	0.852
one-hot + forest (300)	0.895	0.849
raw table + HistGB	0.921	0.869

- ▶ Better, and 2.5 seconds for the fit. Majority baseline 0.759.
- ▶ Needs the category dtype and scikit-learn 1.4 or newer (Colab has it). Make it your first model on any messy table; the cleaning can come second.

Trees & Ensembles: **Feature Engineering**

Move	What it looks like	Who used it
a flag for every filled value	<code>df[c + "_is_nan"] = df[c].isna(),</code> then impute the median fitted on train	NEOAI 2025 Tricy Table solution
log of a skewed target	<code>np.log</code> before fit, <code>np.exp</code> after predict	NEOAI 2025; Day 08 Lab 1
threshold indicators	<code>df["f0_low"] = df["feat_0"] < 500</code>	NEOAI 2025
aggregates over repeated columns	mean, min, max, std and trend (<code>grade_12 - grade_9</code>) of the four yearly grades	HAIO 2025 sample solution
counts	subjects taken, advanced exams taken, loan types	HAIO 2025; OJIA Credit Score
a number hidden in text	owners range to its midpoint; "13 Years and 10 Months" to 166	ONIA Steam; OJIA Credit Score
a list column to many 0/1 columns	<code>str.get_dummies(sep=",")</code>	ONIA Steam (genres), OJIA (loan types)
drop near-zero correlations	<code>X.corrwith(y)</code> ranking, drop the bottom	NEOAI 2025

Every move is a few lines of pandas. None of them changes the model; all of them change what the model can see. The next frames measure the ones the lecture did not.

A 57-Value Column, Three Ways, Two Models

used cars: how Make enters	columns	Ridge MAE	forest MAE
not at all	12	27,807	21,967
one-hot (one 0/1 column per make)	68	20,219	15,282
frequency (how many listings share the make)	13	27,757	15,712
target encoding (the make's mean log price, cross-fitted)	13	21,761	15,420

- **Frequency encoding** replaces the category by its count. It costs one column and carries “common or rare”. The forest extracts nearly everything from it (15,712 against 15,282); the line gets nothing, because a count has no price order.
- **Target encoding** replaces the category by the mean target of its rows. One column, and now the line recovers most of the gap (21,761). The danger is leakage: a rare make's own price leaks into its code. TargetEncoder avoids it by computing each training row's code from the *other* folds.
- **One-hot** wins both here because 57 values fit 4,228 rows. With 5,000 values (a city, a product id) it stops being an option and the two one-column encodings are the move.

```
# frequency: counts learned on the training rows, applied to both tables
counts = X_train["Make"].value_counts()
X_train["make_freq"] = X_train["Make"].map(counts)
X_val["make_freq"] = X_val["Make"].map(counts).fillna(0)      # unseen make ->
0

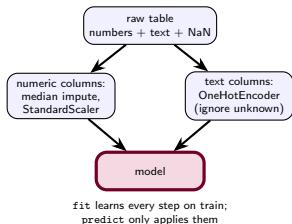
# target encoding: fit_transform on train is cross-fitted (each row coded from
the other folds)
from sklearn.preprocessing import TargetEncoder
enc = TargetEncoder(random_state=0)
X_train["make_te"] = enc.fit_transform(X_train[["Make"]], np.log(y_train))[:,
0]
X_val["make_te"] = enc.transform(X_val[["Make"]])[:, 0]      # plain training
means
```

The leak to avoid

`df["make_te"] = df.groupby("Make")["Price"].transform("mean")` on the whole table codes every row with a mean that includes its own target, and includes the validation rows' targets too. The validation score becomes a lie; the test score stays honest and disappoints. Learn every encoding on the training rows only.

Steam prices, boosting (300 trees, depth 2, rate 0.05)	validation MAE
the 24 columns of Day 10's Lab 2	6.46
plus publisher_count (how many games the publisher has in the training table)	6.35
plus n_genres (how many genres the game lists)	6.44
plus month of release	6.46
plus name_len (characters in the title)	6.47
all four at once	6.40

- ▶ One column out of four paid: the publisher's frequency, a one-line encoding of a 1,600-value text column the lecture's solution dropped. Two did nothing and one hurt a little; together they dilute the winner.
- ▶ So: add one column, score, keep or drop, repeat. The JOAI 2025 gold write-up reports the same discipline from the other side: several hand-made columns lowered its score and were removed.
- ▶ Guesses about which column helps are usually wrong in both directions; the validation number is the only judge.



```

from sklearn.compose import
ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import
OneHotEncoder
num_cols = X.select_dtypes("number").
columns
cat_cols = X.select_dtypes("object").
columns
prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(
        strategy="median"),
        StandardScaler()
    ), num_cols),
    ("cat", OneHotEncoder(handle_unknown="
        ignore"), cat_cols)])
pipe = make_pipeline(pre,
    LogisticRegression(max_iter=2000))
cross_val_score(pipe, X, y, cv=5, scoring=
    "roc_auc")
    
```

Inside `cross_val_score` the imputer, scaler and encoder are refitted on each training fold; `handle_unknown="ignore"` turns a category unseen in train into zeros instead of an error.

Trees & Ensembles: **Olympiad Practice**

Task

A bank table: 44,207 training rows, 27 columns, 8,841 test rows. Target `Credit_Score`: poor (-1 , 31%), standard (0, 53%), good (1, 16%). Score: accuracy on the test rows (80 of 100 points; four counting questions make the rest). The columns are Day 06's: Age is text with underscores ("28_"), Occupation has 3,086 rows of "_____", Payment_Behaviour contains "!@9#%8", Credit_History_Age reads "13 Years and 10 Months", Type_of_Loan is a list, ages run from -500 to 8,697, and there is no NaN anywhere.

- ▶ **Question 1:** in which order do you clean, encode and model, so that the first submission exists within fifteen minutes?
- ▶ **Question 2:** which of today's models needs the least of that cleaning, and what must still be done by hand?
- ▶ **Question 3:** the reference solution reached accuracy 0.79 with a default random forest. Where would you look for the next three points?

Adapted from Olimpiada Nationala de Inteligenta Artificiala (Romania, 2025), *Simulare OJIA 1: Credit Score*; data and a solved notebook in the [jaredliw/ioai-tsp-2025](#) repository.

Solution Idea: the Cleaning Function

```
def clean(d):  
    d = d.copy()  
    for c in ["Age", "Annual_Income", "Num_of_Loan", "Num_of_Delayed_Payment", "  
        Changed_Credit_Limit",  
            "Outstanding_Debt", "Amount_invested_monthly", "Monthly_Balance"]:  
        d[c] = pd.to_numeric(d[c].astype(str).str.replace("_", ""), errors="coerce")  
    years = d["Credit_History_Age"].str.extract(r"(\d+) Years")[0].astype(float)  
    months = d["Credit_History_Age"].str.extract(r"(\d+) Months")[0].astype(float)  
    d["Credit_History_Age"] = years * 12 + months  
    d["Age"] = d["Age"].where((d["Age"] >= 18) & (d["Age"] <= 100))      # impossible ->  
    NaN  
    d["n_loan_types"] = d["Type_of_Loan"].fillna("").str.count(",") + d["Type_of_Loan"].  
        notna()  
    for c in ["Occupation", "Credit_Mix", "Payment_of_Min_Amount", "Payment_Behaviour", "  
        Month"]:  
        d[c] = d[c].replace({"_____": np.nan, "_": np.nan, "!@9#%8": np.nan}).astype("category")  
    return d.drop(columns=["ID", "Customer_ID", "Name", "SSN", "Type_of_Loan"])  
  
hgb = HistGradientBoostingClassifier(categorical_features="from_dtype", max_iter=300,  
    random_state=0)  
cross_val_score(hgb, clean(train), train["Credit_Score"], cv=KFold(5, shuffle=True,  
    random_state=0)).mean()      # 0.754
```

No filling, no scaling, no one-hot. Majority baseline 0.528; this cell gives 0.754.

1. **Order:** numbers first (one character each), the two text-to-number columns, impossible values to NaN, junk categories to NaN, category dtype, drop ids and names; then HistGradientBoosting straight away and upload. Only then tune or engineer.
2. **Least cleaning:** the boosting above needs no filling, no scaling, no one-hot. What stays by hand: the underscores, the “Years and Months” text, the list column, and the junk category values, because no model can read those.
3. **Next points:** aggregate the customer's other months (each Customer_ID has up to 8 rows); tune the rate and the number of trees from a staged curve; try `class_weight="balanced"` only if the metric were balanced (it is plain accuracy, so no).

Skipped on purpose

The same customer sits in training *and* test rows. That changes how you should validate, and which number to believe: Day 13.

Trees & Ensembles: **Check Yourself**

Answer before turning the page.

1. **Definition.** A forest's out-of-bag score is 0.932 and its score on the training rows is 1.000. Which number do you report as the model's expected quality, and why can a forest score 1.000 on rows it trained on?
2. **Mechanism.** A regression tree node holds prices 40, 60, 80 and 100 (thousand SAR). A cut separates $\{40, 60\}$ from $\{80, 100\}$. Compute the node's MSE before the cut and the weighted MSE after it.
3. **Code.** What is wrong with this encoding, and which score will it inflate?

```
df["make_te"] = df.groupby("Make")["Price"].transform("mean")
X_train, X_val, y_train, y_val = train_test_split(df.drop(columns="Price"), df
["Price"], random_state=42)
```

1. Report 0.932. Each training row is predicted by trees that did not see it, so OOB is a held-out score; the forest's trees have unlimited depth and memorise their own rows, so on the training rows every row is in-sample for most trees and the score is meaningless.
2. Mean before: 70; $MSE = \frac{(30^2 + 10^2 + 10^2 + 30^2)}{4} = \frac{900 + 100 + 100 + 900}{4} = 500$. After: left mean 50, $MSE = \frac{10^2 + 10^2}{2} = 100$; right mean 90, MSE 100; weighted $\frac{2 \cdot 100 + 2 \cdot 100}{4} = 100$. The cut removes 80% of the impurity (in squared thousand SAR).
3. The mean is computed on the whole table before the split, so (a) every row's code contains its own price and (b) the validation rows' prices leak into the training rows' codes. The validation MAE will look too good; the test MAE will not. Fit TargetEncoder on the training rows only, after the split, and transform the validation rows.

Trees & Ensembles: **Limitations & What Comes Next**

- ▶ **Trees never extrapolate.** The leaf value is the prediction, so every price beyond the training range is the last step of the staircase. A line (or a log-linear model) is the better tool when the test rows go beyond the training rows.
- ▶ **Early stopping uses rows.** Its 15% comes out of the training set; on a 300-row table that is 45 rows deciding when to stop. Use the staged curve on a cross-validated split instead.
- ▶ **Target encoding is fragile on rare categories.** A make with two listings gets a mean of two prices; TargetEncoder shrinks it towards the global mean, but the forest may still prefer the raw frequency.
- ▶ **Boosting is sensitive to the rate and the trees together.** Change one, re-tune the other.
- ▶ **Feature importance of a boosting model is not evidence** that a column causes anything; Day 13 shows what it is and is not.

The common mistake

Adding ten engineered columns in one cell and keeping them all because the score moved. Add one, measure, decide.

- ▶ **Day 13, self-study, Olympiad practice:** how to validate so that the number you see is the number the judge sees: stratified and grouped folds, out-of-fold predictions, the spread of a score, PCA, feature importance and its traps, and the submission routine.
- ▶ **Week 3:** a model that learns its own features from raw pixels and text, trained by the gradient descent of Day 08, with early stopping and learning rates that will look familiar.

Trees & Ensembles: **Summary**

- ▶ **Regression trees** cut where the weighted MSE drops most; `export_text` shows the rules and the leaf values are the only predictions.
- ▶ **Dials beyond depth:** `min_samples_leaf` lets a tree go deep only where data is dense; `max_features` keeps the forest's trees different.
- ▶ **Out-of-bag:** a free validation score from the rows each tree never saw; more trees never hurt.
- ▶ **Boosting:** halve the rate, double the trees; read the number of trees from `staged_predict` or let `HistGradientBoosting` stop early.
- ▶ **HistGradientBoosting** takes NaN and category columns as they are, and beat the one-hot forest on the Adult census.
- ▶ **Feature engineering:** the checklist from real solutions, three encodings for a big category, measure every added column, and `ColumnTransformer` so nothing leaks.

Trees & Ensembles: **References & Resources**

- [1] Bin Talib, T. *Saudi Arabia Used Cars Dataset*, Kaggle turkibintalib/saudi-arabia-used-cars-dataset. The regression-tree figure and the encoding experiment. <https://www.kaggle.com/datasets/turkibintalib/saudi-arabia-used-cars-dataset>
- [2] Hungarian AI Olympiad (2025). *Online qualifier: University Admissions* and its sample solution (CC BY-NC-SA 4.0). Dial, OOB and checklist material. <https://github.com/Hungarian-AI-Olympiad/HAIO-Hungarian-AI-Olympiad/tree/main/2025/nyari-online>
- [3] Olimpiada-AI (2024/25). *Predicting Game Prices on the Steam Platform*, problem-proposal example with reference solutions. Boosting curves and the added-column experiment. <https://github.com/Olimpiada-AI/Propunere-probleme-si-solutii>
- [4] Becker, B. and Kohavi, R. (1996). *Adult* data set, Kaggle uciml/adult-census-income. The native-NaN boosting comparison. <https://www.kaggle.com/datasets/uciml/adult-census-income>
- [5] Olimpiada Nationala de Inteligenta Artificiala, Romania (2025). *Simulare OJIA 1: Credit Score*; data and solved notebook in <https://github.com/jaredliw/ioai-tsp-2025> (onia-romania-2025/simulare-ojia-1/credit-score). The practice task.

- [6] Sandei, S. (2025). *roai-solved*, NEOAI 2025 Tricky Table solution (NaN flags, median imputation, log target, threshold indicators).
<https://github.com/stefanasandei/roai-solved/tree/main/international-contests/neoai/2025>
- [7] rayramy04 (2025). *JOAI 2025 solutions* (gold medal write-up: hand-made columns that lowered the score were removed). <https://github.com/rayramy04/joai2025-solutions>
- [8] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR* 12. HistGradientBoosting, TargetEncoder, ColumnTransformer, staged_predict, oob_score. <https://scikit-learn.org>
- [9] All plots: `slides-builder/scripts/day12_trees_figures.py` (matplotlib, real data only).